

MODUL PRAKTIKUM

PRAKTIKUM DASAR PEMROGRAMAN

Modul 6

Perulangan Bersarang, Pola, Tabel, dan Debugging Program



Disusun oleh:

Dr. Desta Sandya Prasvita, S.Komp., M.Kom.

PROGRAM STUDI S1 INFORMATIKA
UPN VETERAN JAKARTA
2026

Modul ini melanjutkan penggunaan struktur perulangan pada Modul 5. Fokus praktikum adalah mengimplementasikan perulangan bersarang untuk memproses data bertingkat, membentuk pola, menghasilkan tabel, serta menelusuri dan memperbaiki kesalahan pada program yang menggunakan kombinasi seleksi dan perulangan.

I. Pendahuluan Modul

1. Identitas Modul

Komponen	Keterangan
Mata Kuliah	Praktikum Dasar Pemrograman
Program Studi	S1 Informatika
Semester	1
Modul	6
Judul	Perulangan Bersarang, Pola, Tabel, dan Debugging Program
Bahasa Pemrograman	C
Alokasi Waktu	1 × 170 menit
Metode	Demonstrasi, latihan terbimbing, tracing, eksperimen, debugging workshop, dan latihan mandiri
Prasyarat	Modul 1–5

2. Indikator Keberhasilan

Mahasiswa dinyatakan berhasil apabila mampu:

- menulis nested loop dengan struktur yang benar;
- menjelaskan peran counter luar dan counter dalam;
- menghitung jumlah iterasi nested loop;
- menampilkan pola dengan jumlah baris dan kolom sesuai input;
- membuat tabel perkalian atau tabel data sederhana;
- menggunakan kondisi dalam perulangan untuk mengatur isi tabel;
- menginisialisasi accumulator pada tingkat yang tepat;
- membedakan total per baris dengan total keseluruhan;
- menemukan kesalahan melalui trace table;
- menggunakan warning compiler sebagai sumber informasi;
- menggunakan breakpoint dan variable watch apabila tersedia;
- menguji program menggunakan data normal, batas, dan tidak valid;
- menjelaskan proses perbaikan yang dilakukan.

II. Persiapan Praktikum

1. Persiapan Folder Praktikum

Buat struktur folder berikut:

```
Praktikum_Dasar_Pemrograman/
├── Modul_06/
│   ├── latihan_01.c
│   ├── latihan_02.c
│   ├── latihan_03.c
│   └── latihan_04.c
```

```

├─ tracing.c
├─ eksperimen.c
├─ debugging_01.c
├─ debugging_02.c
└─ tugas_modul_06.c

```

II. Konsep Dasar

1. Konsep Perulangan Bersarang

Perulangan bersarang atau nested loop adalah perulangan yang berada di dalam perulangan lain.

Bentuk umum:

```

for (inisialisasi_luar; kondisi_luar; update_luar)
{
    for (inisialisasi_dalam; kondisi_dalam; update_dalam)
    {
        pernyataan;
    }
}

```

Contoh:

```

#include <stdio.h>

int main(void)
{
    int baris;
    int kolom;

    for (baris = 1; baris <= 3; baris++)
    {
        for (kolom = 1; kolom <= 4; kolom++)
        {
            printf("(%d,%d) ", baris, kolom);
        }

        printf("\n");
    }

    return 0;
}

```

Keluaran:

```

(1,1) (1,2) (1,3) (1,4)
(2,1) (2,2) (2,3) (2,4)
(3,1) (3,2) (3,3) (3,4)

```

2. Perulangan Luar dan Dalam

Pada contoh sebelumnya:

```
for (baris = 1; baris <= 3; baris++)
```

disebut perulangan luar. Perulangan luar biasanya mengatur:

- jumlah baris;
- jumlah kelompok;
- jumlah mahasiswa;
- jumlah hari;
- jumlah bagian utama.

Sementara itu:

```
for (kolom = 1; kolom <= 4; kolom++)
```

disebut perulangan dalam. Perulangan dalam biasanya mengatur:

- jumlah kolom;
- jumlah data dalam setiap kelompok;
- jumlah mata kuliah per mahasiswa;
- jumlah produk per hari;
- jumlah elemen pada setiap baris.

Untuk setiap satu iterasi perulangan luar, perulangan dalam dijalankan dari awal sampai selesai.

3. Urutan Eksekusi Nested Loop

Perhatikan:

```
for (baris = 1; baris <= 2; baris++)
{
    for (kolom = 1; kolom <= 3; kolom++)
    {
        printf("%d%d ", baris, kolom);
    }
}
```

Urutan eksekusi:

```
baris = 1
    kolom = 1 → tampilkan 11
    kolom = 2 → tampilkan 12
    kolom = 3 → tampilkan 13

baris = 2
    kolom = 1 → tampilkan 21
    kolom = 2 → tampilkan 22
    kolom = 3 → tampilkan 23
```

Keluaran:

```
11 12 13 21 22 23
```

Apabila ditambahkan `printf("\n");` setelah perulangan dalam:

```
for (baris = 1; baris <= 2; baris++)
{
    for (kolom = 1; kolom <= 3; kolom++)
    {
        printf("%d%d ", baris, kolom);
    }

    printf("\n");
}
```

keluarannya menjadi:

```
11 12 13
21 22 23
```

Posisi `printf("\n")` sangat menentukan bentuk keluaran.

4. Menghitung Jumlah Iterasi

Apabila perulangan luar berjalan sebanyak m kali dan perulangan dalam selalu berjalan sebanyak n kali, jumlah eksekusi blok terdalam adalah:

$$m \times n$$

Contoh:

```
for (baris = 1; baris <= 3; baris++)
{
    for (kolom = 1; kolom <= 4; kolom++)
    {
        printf("*");
    }
}
```

Perulangan luar: 3 kali. Perulangan dalam pada setiap baris: 4 kali.

Jumlah `printf("*")`:

$$3 \times 4 = 12 \text{ kali}$$

Jumlah iterasi yang berubah

Perhatikan:

```
for (baris = 1; baris <= 4; baris++)
{
    for (kolom = 1; kolom <= baris; kolom++)
    {
        printf("*");
    }
}
```

```
    printf("\n");
}
```

Jumlah iterasi per baris:

Baris	Banyak iterasi dalam
1	1
2	2
3	3
4	4

Total: $1 + 2 + 3 + 4 = 10$

Keluaran:

```
*
**
***
****
```

5. Nested Loop dengan Struktur Berbeda

Perulangan bersarang tidak harus menggunakan jenis yang sama.

for di dalam while

```
int baris = 1;
int kolom;

while (baris <= 3)
{
    for (kolom = 1; kolom <= 4; kolom++)
    {
        printf("*");
    }

    printf("\n");
    baris++;
}
```

do-while di dalam for

```
int baris;
int kolom;

for (baris = 1; baris <= 3; baris++)
{
    kolom = 1;

    do
```

```

{
    printf("%d ", kolom);
    kolom++;
}
while (kolom <= 3);

printf("\n");
}

```

Meskipun diperbolehkan, gunakan kombinasi yang paling mudah dipahami.

Untuk pola dengan jumlah baris dan kolom yang diketahui, dua buah for biasanya paling jelas.

6. Membentuk Pola Persegi

Program berikut menampilkan pola persegi berukuran 4×4 .

```

#include <stdio.h>

int main(void)
{
    int baris;
    int kolom;

    for (baris = 1; baris <= 4; baris++)
    {
        for (kolom = 1; kolom <= 4; kolom++)
        {
            printf("* ");
        }

        printf("\n");
    }

    return 0;
}

```

Keluaran:

```

* * * *
* * * *
* * * *
* * * *

```

Perulangan luar menentukan jumlah baris, sedangkan perulangan dalam menentukan jumlah simbol dalam setiap baris.

7. Membentuk Pola Segitiga Naik

```
#include <stdio.h>

int main(void)
{
    int baris;
    int kolom;

    for (baris = 1; baris <= 5; baris++)
    {
        for (kolom = 1; kolom <= baris; kolom++)
        {
            printf("* ");
        }

        printf("\n");
    }

    return 0;
}
```

Keluaran:

```
*
* *
* * *
* * * *
* * * * *
```

Perhatikan kondisi:

```
kolom <= baris
```

Jumlah simbol bergantung pada nomor baris.

8. Membentuk Pola Segitiga Turun

```
#include <stdio.h>

int main(void)
{
    int baris;
    int kolom;

    for (baris = 5; baris >= 1; baris--)
    {
        for (kolom = 1; kolom <= baris; kolom++)
        {
            printf("* ");
        }
    }
}
```

```

        printf("\n");
    }

    return 0;
}

```

Keluaran:

```

* * * * *
* * * *
* * *
* *
*

```

9. Membentuk Pola Bilangan

Contoh:

```

for (baris = 1; baris <= 5; baris++)
{
    for (kolom = 1; kolom <= baris; kolom++)
    {
        printf("%d ", kolom);
    }

    printf("\n");
}

```

Keluaran:

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

```

Apabila yang ditampilkan adalah baris:

```

printf("%d ", baris);

```

keluaran menjadi:

```

1
2 2
3 3 3
4 4 4 4
5 5 5 5 5

```

Mahasiswa harus membedakan kapan menggunakan counter baris dan kapan menggunakan counter kolom.

10. Pola Menggunakan Kondisi

Perhatikan pola berikut:

```
* - * - *
- * - * -
* - * - *
- * - * -
* - * - *
```

Pola dapat dibentuk menggunakan:

```
#include <stdio.h>

int main(void)
{
    int baris;
    int kolom;

    for (baris = 1; baris <= 5; baris++)
    {
        for (kolom = 1; kolom <= 5; kolom++)
        {
            if ((baris + kolom) % 2 == 0)
            {
                printf("* ");
            }
            else
            {
                printf("- ");
            }
        }

        printf("\n");
    }

    return 0;
}
```

Konsep yang digunakan:

- nested loop untuk menentukan posisi;
- seleksi untuk menentukan karakter;
- modulo untuk membentuk pola bergantian.

11. Membuat Tabel Perkalian

```
#include <stdio.h>

int main(void)
{
    int baris;
```

```

int kolom;

printf("    ");

for (kolom = 1; kolom <= 5; kolom++)
{
    printf("%4d", kolom);
}

printf("\n");

for (baris = 1; baris <= 5; baris++)
{
    printf("%4d", baris);

    for (kolom = 1; kolom <= 5; kolom++)
    {
        printf("%4d", baris * kolom);
    }

    printf("\n");
}

return 0;
}

```

Keluaran:

	1	2	3	4	5
1	1	2	3	4	5
2	2	4	6	8	10
3	3	6	9	12	15
4	4	8	12	16	20
5	5	10	15	20	25

Format %4d berarti menyediakan lebar minimal empat karakter agar kolom lebih rapi.

12. Memproses Data Bertingkat

Nested loop tidak hanya digunakan untuk pola. Struktur ini juga digunakan untuk data yang terdiri atas kelompok dan anggota.

Contoh:

- beberapa mahasiswa, masing-masing memiliki beberapa nilai;
- beberapa hari, masing-masing memiliki beberapa transaksi;
- beberapa kelas, masing-masing memiliki beberapa siswa;
- beberapa cabang, masing-masing memiliki beberapa produk;
- beberapa baris, masing-masing memiliki beberapa kolom.

Contoh program:

```

#include <stdio.h>

int main(void)
{
    int mahasiswa;
    int tugas;
    int jumlah_mahasiswa;
    int jumlah_tugas;

    double nilai;
    double total_mahasiswa;
    double rata_rata_mahasiswa;
    double total_keseluruhan = 0.0;

    printf("Masukkan jumlah mahasiswa: ");
    scanf("%d", &jumlah_mahasiswa);

    printf("Masukkan jumlah tugas: ");
    scanf("%d", &jumlah_tugas);

    for (mahasiswa = 1;
        mahasiswa <= jumlah_mahasiswa;
        mahasiswa++)
    {
        total_mahasiswa = 0.0;

        printf("\nMahasiswa ke-%d\n", mahasiswa);

        for (tugas = 1; tugas <= jumlah_tugas; tugas++)
        {
            printf("Nilai tugas ke-%d: ", tugas);
            scanf("%lf", &nilai);

            total_mahasiswa += nilai;
            total_keseluruhan += nilai;
        }

        rata_rata_mahasiswa =
            total_mahasiswa / jumlah_tugas;

        printf("Rata-rata mahasiswa ke-%d: %.2f\n",
            mahasiswa,
            rata_rata_mahasiswa);
    }

    printf("\nTotal keseluruhan nilai: %.2f\n",
        total_keseluruhan);
}

```

```
    return 0;
}
```

13. Posisi Inisialisasi Accumulator

Perhatikan:

```
for (mahasiswa = 1;
    mahasiswa <= jumlah_mahasiswa;
    mahasiswa++)
{
    total_mahasiswa = 0.0;

    for (tugas = 1; tugas <= jumlah_tugas; tugas++)
    {
        scanf("%lf", &nilai);
        total_mahasiswa += nilai;
    }
}
```

Variabel `total_mahasiswa` harus diinisialisasi kembali pada awal setiap mahasiswa.

Apabila inisialisasi ditempatkan di luar perulangan mahasiswa:

```
double total_mahasiswa = 0.0;

for (mahasiswa = 1;
    mahasiswa <= jumlah_mahasiswa;
    mahasiswa++)
{
    for (tugas = 1; tugas <= jumlah_tugas; tugas++)
    {
        total_mahasiswa += nilai;
    }
}
```

total mahasiswa kedua akan ikut membawa nilai mahasiswa pertama.

Pedoman

Jenis hasil	Lokasi inisialisasi
Total per baris/per mahasiswa/per hari	Di dalam perulangan luar, sebelum perulangan dalam
Total keseluruhan	Sebelum perulangan luar
Counter per kelompok	Di dalam perulangan luar
Counter seluruh data	Sebelum perulangan luar

14. Penggunaan break pada Nested Loop

Perhatikan:

```
for (baris = 1; baris <= 3; baris++)
{
    for (kolom = 1; kolom <= 5; kolom++)
```

```

{
    if (kolom == 3)
    {
        break;
    }

    printf("(%d,%d) ", baris, kolom);
}

printf("\n");
}

```

Keluaran:

```

(1,1) (1,2)
(2,1) (2,2)
(3,1) (3,2)

```

break hanya menghentikan perulangan terdekat, yaitu perulangan kolom.

Perulangan baris tetap dilanjutkan.

Untuk menghentikan seluruh nested loop, diperlukan rancangan tambahan, misalnya:

- variabel penanda;
- pengaturan kondisi loop;
- pemindahan proses ke fungsi pada materi berikutnya.

15. Penggunaan continue pada Nested Loop

```

for (baris = 1; baris <= 3; baris++)
{
    for (kolom = 1; kolom <= 4; kolom++)
    {
        if (kolom == 2)
        {
            continue;
        }

        printf("(%d,%d) ", baris, kolom);
    }

    printf("\n");
}

```

Keluaran:

```

(1,1) (1,3) (1,4)
(2,1) (2,3) (2,4)
(3,1) (3,3) (3,4)

```

continue hanya memengaruhi iterasi perulangan tempat ia berada.

Gunakan break dan continue secara terbatas agar alur program tetap mudah ditelusuri.

16. Konsep Debugging

Debugging adalah proses menemukan, memahami, dan memperbaiki kesalahan pada program.

Debugging bukan sekadar mencoba mengubah kode sampai program berjalan. Debugging yang baik dilakukan secara sistematis:

1. mereproduksi masalah;
2. membaca pesan error atau warning;
3. menentukan lokasi yang kemungkinan bermasalah;
4. membuat hipotesis penyebab;
5. memeriksa nilai variabel;
6. memperbaiki satu bagian;
7. mengompilasi dan menguji kembali;
8. memastikan perbaikan tidak menimbulkan masalah baru.

17. Jenis Kesalahan Program

a. *Syntax Error*

Terjadi apabila kode tidak mengikuti aturan bahasa C.

Contoh:

```
for (i = 1; i <= 5; i++
{
    printf("%d\n", i);
}
```

Kesalahan: kurung penutup pada for tidak lengkap.

b. *Compilation Error*

Program tidak dapat diterjemahkan oleh compiler.

Contoh penyebab:

- titik koma hilang;
- nama variabel belum dideklarasikan;
- kurung kurawal tidak seimbang;
- format source code tidak sesuai sintaks;
- penggunaan operator yang tidak valid.

c. *Warning*

Compiler menemukan bagian yang mencurigakan, tetapi kompilasi mungkin tetap berhasil.

Contoh:

```
int total;

printf("%d\n", total);
```

Variabel total belum diinisialisasi.

Warning harus dianalisis karena dapat menunjukkan logical error atau undefined behavior.

d. Runtime Error

Program berhasil dikompilasi tetapi mengalami masalah saat dijalankan.

Contoh:

- pembagian dengan nol;
- input yang tidak sesuai;
- proses mengakses data yang tidak valid;
- penggunaan memori yang salah pada modul berikutnya.

e. Logical Error

Program dapat dikompilasi dan dijalankan, tetapi hasilnya salah.

Contoh:

```
total = nilai;
```

ketika seharusnya:

```
total += nilai;
```

Logical error sering lebih sulit ditemukan karena compiler tidak selalu memberikan pesan.

f. Infinite Loop

Program terus berjalan karena kondisi berhenti tidak pernah tercapai.

```
int i = 1;

while (i <= 5)
{
    printf("%d\n", i);
}
```

Nilai i tidak berubah.

g. Off-by-One Error

Perulangan berjalan satu kali terlalu banyak atau terlalu sedikit.

```
for (i = 1; i < 5; i++)
```

menghasilkan empat iterasi, bukan lima.

18. Teknik Debugging Dasar**a. Membaca pesan pertama dari compiler**

Satu kesalahan awal dapat menyebabkan banyak pesan lanjutan. Perbaiki pesan pertama terlebih dahulu, lalu kompilasi ulang.

b. Memeriksa baris sebelumnya

Compiler dapat menunjukkan kesalahan pada baris setelah lokasi sebenarnya.

Contoh:

```
printf("Baris pertama\n")
printf("Baris kedua\n");
```

Error mungkin ditunjukkan pada printf kedua, padahal penyebabnya adalah titik koma yang hilang pada baris pertama.

c. Menampilkan nilai sementara

```
printf("DEBUG: baris=%d, kolom=%d\n",
      baris,
      kolom);
```

Pesan sementara membantu melihat alur program. Hapus atau nonaktifkan setelah debugging selesai.

d. Menggunakan trace table

Catat nilai setiap variabel untuk setiap iterasi.

e. Membagi masalah

Apabila program besar, uji bagian-bagian kecil:

- validasi input;
- perulangan luar;
- perulangan dalam;
- accumulator;
- format output.

f. Menggunakan breakpoint

Breakpoint menghentikan program sementara pada baris tertentu.

Ketika program berhenti, mahasiswa dapat:

- melihat nilai variabel;
- menjalankan satu baris;
- masuk ke blok tertentu;
- melanjutkan eksekusi.

g. Variable watch

Variable watch digunakan untuk mengamati perubahan nilai variabel selama program berjalan.

Variabel yang dapat diamati:

```
baris
kolom
total_baris
total_keseluruhan
nilai
```

IV. Praktikum Terbimbing

1. Praktikum Terbimbing 1 — Membentuk Pola Segitiga

Buat berkas: latihan_01.c. Program:

```
#include <stdio.h>

int main(void)
{
```

```

int tinggi;
int baris;
int kolom;

do
{
    printf("Masukkan tinggi pola 1-20: ");
    scanf("%d", &tinggi);

    if ((tinggi < 1) || (tinggi > 20))
    {
        printf("Tinggi harus berada pada rentang 1-20.\n");
    }
}
while ((tinggi < 1) || (tinggi > 20));

printf("\nPOLA SEGITIGA\n");

for (baris = 1; baris <= tinggi; baris++)
{
    for (kolom = 1; kolom <= baris; kolom++)
    {
        printf("* ");
    }

    printf("\n");
}

return 0;
}

```

Contoh

Input: 4

Keluaran:

```

*
* *
* * *
* * * *

```

Pertanyaan analisis

1. Apa fungsi perulangan luar?
2. Apa fungsi perulangan dalam?
3. Mengapa kondisi perulangan dalam menggunakan kolom <= baris?
4. Berapa kali printf("* ") dijalankan untuk tinggi 4?
5. Apa hasil jika menggunakan kolom < baris?
6. Apa hasil jika printf("\n") dipindahkan ke dalam perulangan kolom?
7. Apa hasil jika counter kolom tidak diinisialisasi kembali?

8. Mengapa tinggi dibatasi maksimal 20?

2. Praktikum Terbimbing 2 — Membuat Tabel Perkalian

Buat berkas: latihan_02.c. Program:

```
#include <stdio.h>

int main(void)
{
    int batas;
    int baris;
    int kolom;

    do
    {
        printf("Masukkan batas tabel 1-12: ");
        scanf("%d", &batas);

        if ((batas < 1) || (batas > 12))
        {
            printf("Batas harus berada pada rentang 1-12.\n");
        }
    }
    while ((batas < 1) || (batas > 12));

    printf("\nTABEL PERKALIAN\n\n");

    printf("%5s", "x");

    for (kolom = 1; kolom <= batas; kolom++)
    {
        printf("%5d", kolom);
    }

    printf("\n");

    for (baris = 1; baris <= batas; baris++)
    {
        printf("%5d", baris);

        for (kolom = 1; kolom <= batas; kolom++)
        {
            printf("%5d", baris * kolom);
        }

        printf("\n");
    }
}
```

```
    return 0;
}
```

Contoh keluaran untuk batas 4

x	1	2	3	4
1	1	2	3	4
2	2	4	6	8
3	3	6	9	12
4	4	8	12	16

Pertanyaan analisis

1. Mengapa bagian header kolom dibuat menggunakan perulangan terpisah?
2. Apa fungsi %5d?
3. Berapa kali perkalian dilakukan untuk batas 4?
4. Berapa kali perkalian dilakukan untuk batas 10?
5. Mengapa nilai kolom dimulai kembali dari 1 pada setiap baris?
6. Apa akibatnya apabila printf("\n") dihapus?
7. Bagaimana cara menampilkan tabel dari 0 sampai batas?
8. Apakah tabel dapat dibuat menggunakan while?

3. Praktikum Terbimbing 3 — Rekap Nilai Beberapa Mahasiswa

Buat berkas: latihan_03.c. Program menerima beberapa mahasiswa. Setiap mahasiswa memiliki beberapa nilai tugas.

```
#include <stdio.h>

int main(void)
{
    int jumlah_mahasiswa;
    int jumlah_tugas;
    int mahasiswa;
    int tugas;

    double nilai;
    double total_mahasiswa;
    double rata_rata_mahasiswa;
    double total_keseluruhan = 0.0;
    double rata_rata_keseluruhan;

    int jumlah_nilai = 0;

    do
    {
        printf("Masukkan jumlah mahasiswa 1-30: ");
        scanf("%d", &jumlah_mahasiswa);
```

```

}
while ((jumlah_mahasiswa < 1) ||
       (jumlah_mahasiswa > 30));

do
{
    printf("Masukkan jumlah tugas 1-10: ");
    scanf("%d", &jumlah_tugas);
}
while ((jumlah_tugas < 1) ||
       (jumlah_tugas > 10));

for (mahasiswa = 1;
     mahasiswa <= jumlah_mahasiswa;
     mahasiswa++)
{
    total_mahasiswa = 0.0;

    printf("\nMahasiswa ke-%d\n", mahasiswa);

    for (tugas = 1; tugas <= jumlah_tugas; tugas++)
    {
        do
        {
            printf("Nilai tugas ke-%d: ", tugas);
            scanf("%lf", &nilai);

            if ((nilai < 0.0) || (nilai > 100.0))
            {
                printf("Nilai harus berada pada rentang 0-100.\n");
            }
        }
        while ((nilai < 0.0) || (nilai > 100.0));

        total_mahasiswa += nilai;
        total_keseluruhan += nilai;
        jumlah_nilai++;
    }

    rata_rata_mahasiswa =
        total_mahasiswa / jumlah_tugas;

    printf("Total mahasiswa ke-%d      : %.2f\n",
           mahasiswa,
           total_mahasiswa);

    printf("Rata-rata mahasiswa ke-%d: %.2f\n",
           mahasiswa,

```

```

        rata_rata_mahasiswa);
    }

    rata_rata_keseluruhan =
        total_keseluruhan / jumlah_nilai;

    printf("\nREKAP KESELURUHAN\n");
    printf("Jumlah nilai      : %d\n", jumlah_nilai);
    printf("Total keseluruhan : %.2f\n", total_keseluruhan);
    printf("Rata-rata umum      : %.2f\n", rata_rata_keseluruhan);

    return 0;
}

```

Contoh struktur data

```

Mahasiswa 1
  Tugas 1
  Tugas 2
  Tugas 3

Mahasiswa 2
  Tugas 1
  Tugas 2
  Tugas 3

```

Pertanyaan analisis

1. Mengapa total_mahasiswa diinisialisasi di dalam perulangan mahasiswa?
2. Mengapa total_keseluruhan diinisialisasi sebelum perulangan mahasiswa?
3. Berapa jumlah input nilai apabila ada 4 mahasiswa dan 3 tugas?
4. Apa fungsi jumlah_nilai?
5. Apakah jumlah_nilai dapat dihitung melalui perkalian?
6. Apa akibatnya apabila total_mahasiswa tidak diatur kembali menjadi 0?
7. Mengapa validasi nilai menggunakan do-while?
8. Bagaimana cara menentukan mahasiswa dengan rata-rata tertinggi tanpa menggunakan array?

4. Praktikum Terbimbing 4 — Menentukan Posisi pada Tabel

Buat berkas: latihan_04.c. Program menampilkan papan 5×5 . Posisi tertentu ditandai dengan X, sedangkan posisi lainnya menggunakan titik.

```

#include <stdio.h>

int main(void)
{
    int baris_target;
    int kolom_target;
    int baris;

```

```

int kolom;

do
{
    printf("Masukkan baris target 1-5: ");
    scanf("%d", &baris_target);
}
while ((baris_target < 1) || (baris_target > 5));

do
{
    printf("Masukkan kolom target 1-5: ");
    scanf("%d", &kolom_target);
}
while ((kolom_target < 1) || (kolom_target > 5));

printf("\nPAPAN 5 x 5\n");

for (baris = 1; baris <= 5; baris++)
{
    for (kolom = 1; kolom <= 5; kolom++)
    {
        if ((baris == baris_target) &&
            (kolom == kolom_target))
        {
            printf("X ");
        }
        else
        {
            printf(". ");
        }
    }

    printf("\n");
}

return 0;
}

```

Contoh input: baris = 3, kolom = 4. Keluaran:

```

. . . . .
. . . . .
. . . X .
. . . . .
. . . . .

```

Pertanyaan analisis

1. Mengapa digunakan operator &&?
2. Apa yang terjadi apabila operator diganti menjadi ||?

3. Berapa kali kondisi diperiksa?
4. Apakah perulangan dapat dihentikan setelah X ditampilkan?
5. Mengapa tetap diperlukan perulangan sampai seluruh papan selesai?
6. Bagaimana menampilkan diagonal utama menggunakan kondisi baris == kolom?
7. Bagaimana menampilkan bingkai papan?

V. Eksperimen dan Tracing

1. Latihan Tracing

Buat berkas: tracing.c. Jangan langsung menjalankan program berikut.

```
#include <stdio.h>

int main(void)
{
    int baris;
    int kolom;
    int total_baris;
    int total_keseluruhan = 0;

    for (baris = 1; baris <= 3; baris++)
    {
        total_baris = 0;

        for (kolom = 1; kolom <= 3; kolom++)
        {
            total_baris += baris * kolom;
        }

        total_keseluruhan += total_baris;

        printf("Total baris %d: %d\n",
               baris,
               total_baris);
    }

    printf("Total keseluruhan: %d\n",
           total_keseluruhan);

    return 0;
}
```

Lengkapi trace table berikut.

Baris	Kolom	baris * kolom	total_baris	total_keseluruhan
1	1			0
1	2			0
1	3			0

Setelah baris 1	–	–		
2	1			
2	2			
2	3			
Setelah baris 2	–	–		
3	1			
3	2			
3	3			
Setelah baris 3	–	–		

Setelah tracing:

- prediksi total setiap baris;
- prediksi total keseluruhan;
- jalankan program;
- bandingkan hasil;
- jelaskan mengapa total_baris harus diatur menjadi nol pada awal setiap baris.

2. Eksperimen

Buat berkas: eksperimen.c. Gunakan program dasar:

```
#include <stdio.h>

int main(void)
{
    int baris;
    int kolom;

    for (baris = 1; baris <= 3; baris++)
    {
        for (kolom = 1; kolom <= 4; kolom++)
        {
            printf("* ");
        }

        printf("\n");
    }

    return 0;
}
```

Lakukan eksperimen berikut satu per satu.

No.	Perubahan	Prediksi	Hasil Aktual
1	Pindahkan printf("\n") ke dalam loop kolom		

2	Ubah batas baris menjadi 4		
3	Ubah batas kolom menjadi kolom $\leq$ baris		
4	Ubah batas kolom menjadi kolom $<$ baris		
5	Gunakan baris * kolom sebagai output		
6	Tambahkan break ketika kolom $==$ 3		
7	Tambahkan continue ketika kolom $==$ 2		
8	Hapus inisialisasi kolom = 1 dari loop dalam		
9	Ubah update kolom menjadi kolom--		
10	Tambahkan kondisi untuk menampilkan diagonal		

Untuk setiap eksperimen:

- tuliskan prediksi;
- periksa risiko infinite loop;
- kompilasi dengan warning aktif;
- jalankan apabila aman;
- catat hasil;
- jelaskan penyebab;
- kembalikan program ke bentuk awal.

VI. Debugging Challenge

1. Debugging Challenge 1 — Pola yang Tidak Sesuai

Program berikut seharusnya menampilkan:

```
*
**
***
****
*****
```

Namun, program memiliki beberapa kesalahan.

```
#include <stdio.h>

int main(void)
{
    int baris;
    int kolom;

    for (baris = 1; baris < 5; baris++)
    {
        for (kolom = 1; kolom <= 5; kolom--)
        {
            printf("*");
        }
    }
}
```

```

    }

    printf("\n");
}

return 0;
}

```

Tugas

1. Jangan langsung menjalankan program.
2. Identifikasi risiko infinite loop.
3. Perbaiki batas perulangan luar.
4. Perbaiki update perulangan dalam.
5. Perbaiki kondisi agar jumlah simbol mengikuti nomor baris.
6. Kompilasi ulang.
7. Jalankan dan bandingkan dengan pola yang diharapkan.

Tabel Debugging

No.	Bagian Bermasalah	Jenis Kesalahan	Penyebab	Perbaikan
1				
2				
3				

2. Debugging Challenge 2 — Rekap Penjualan yang Salah

Program berikut dimaksudkan untuk mencatat penjualan tiga hari. Setiap hari memiliki empat transaksi.

```

#include <stdio.h>

int main(void)
{
    int hari;
    int transaksi;

    double nilai_transaksi;
    double total_harian = 0.0;
    double total_keseluruhan;

    for (hari = 1; hari <= 3; hari++)
    {
        printf("\nHari ke-%d\n", hari);

        for (transaksi = 1;
            transaksi < 4;
            transaksi++)
        {
            printf("Transaksi ke-%d: ", transaksi);

```

```

        scanf("%d", &nilai_transaksi);

        total_harian = nilai_transaksi;
    }

    printf("Total hari ke-%d: %.2f\n",
           hari,
           total_harian);

    total_keseluruhan =
        total_keseluruhan + total_harian;
}

printf("\nTotal keseluruhan: %.2f\n",
       total_keseluruhan);

return 0;
}

```

Kesalahan yang Perlu Ditemukan

- total_keseluruhan belum diinisialisasi;
- total_harian diinisialisasi pada lokasi yang tidak tepat;
- jumlah transaksi hanya tiga karena menggunakan < 4;
- format specifier scanf() tidak sesuai tipe double;
- total_harian ditimpa, bukan ditambahkan;
- program belum memvalidasi transaksi negatif;
- hasil harian berikutnya membawa total hari sebelumnya.

Tugas

1. Kompilasi dengan warning aktif.
2. Catat warning yang muncul.
3. Perbaiki satu kesalahan pada satu waktu.
4. Uji dengan data berikut: Hari 1: 10, 20, 30, 40; Hari 2: 5, 5, 5, 5; Hari 3: 100, 0, 50, 50.

Expected output:

```

Total hari 1 = 100
Total hari 2 = 20
Total hari 3 = 200
Total keseluruhan = 320

```

Tabel Debugging

No.	Bagian Bermasalah	Error/Warning	Penyebab	Perbaikan
1				
2				
3				
4				

5				
6				

VII. Latihan Mandiri

1. Level 1 — Persegi Panjang Karakter

Buat program yang menerima jumlah baris, jumlah kolom, dan satu karakter. Program menampilkan persegi panjang menggunakan karakter tersebut.

Contoh:

```
Masukkan jumlah baris : 3
Masukkan jumlah kolom : 5
Masukkan karakter      : #

# # # # #
# # # # #
# # # # #
```

Ketentuan:

- baris berada pada rentang 1–20;
- kolom berada pada rentang 1–20;
- input divalidasi menggunakan do-while;
- gunakan nested for.

2. Level 2 — Pola Segitiga Bilangan

Buat program yang menghasilkan pola:

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

Batas pola ditentukan oleh pengguna.

Pertanyaan: variabel mana yang perlu ditampilkan? apakah menggunakan baris atau kolom? berapa jumlah seluruh angka untuk tinggi n?

3. Level 3 — Tabel Penjumlahan

Buat program yang menampilkan tabel penjumlahan dari 1 sampai batas.

Contoh batas 4:

+	1	2	3	4
1	2	3	4	5
2	3	4	5	6
3	4	5	6	7
4	5	6	7	8

Ketentuan:

- batas 1–15;

- gunakan format lebar kolom;
- gunakan nested loop;
- tampilkan header baris dan kolom.

4. Challenge — Pola Piramida

Buat program yang menghasilkan:

```
*
***
*****
*****
*****
```

Untuk tinggi 5.

Petunjuk: pada setiap baris diperlukan sejumlah spasi dan sejumlah simbol.

Rumus:

```
jumlah spasi = tinggi - baris
jumlah simbol = 2 × baris - 1
```

Gunakan dua perulangan dalam: satu untuk spasi, satu untuk simbol.

5. Bilangan Genap dalam Tabel Perkalian

Keluaran program pada soal ini akan diperiksa secara ketat: harus sama persis karakter demi karakter dengan Contoh Output.

Sebuah aplikasi latihan matematika ingin menganalisis tabel perkalian. Buat program C yang menerima jumlah baris dan jumlah kolom sebuah tabel perkalian, lalu menghitung berapa banyak hasil kali yang bernilai genap menggunakan nested loop.

Format Input

Dua baris bilangan bulat: jumlah baris (n) dan jumlah kolom (m).

Batasan

- $1 \leq n \leq 20$;
- $1 \leq m \leq 20$.

Contoh 1

Input:

```
3
4
```

Output:

```
=====
      BILANGAN GENAP DALAM TABEL
=====
Jumlah baris   : 3
Jumlah kolom   : 4
-----
Total sel      : 12
```

Bilangan genap : 8

=====

Contoh 2

Input:

5

5

Output:

```
=====
      BILANGAN GENAP DALAM TABEL
=====
Jumlah baris   : 5
Jumlah kolom   : 5
-----
Total sel      : 25
Bilangan genap : 16
=====
```

Contoh 3

Input:

1

10

Output:

```
=====
      BILANGAN GENAP DALAM TABEL
=====
Jumlah baris   : 1
Jumlah kolom   : 10
-----
Total sel      : 10
Bilangan genap : 5
=====
```

Petunjuk Pendekatan

- gunakan nested for dengan counter baris (1 sampai n) dan kolom (1 sampai m);
- setiap sel bernilai baris * kolom, seperti tabel perkalian biasa;
- gunakan counter genap yang bertambah setiap kali $(\text{baris} * \text{kolom}) \% 2 == 0$;
- total sel dapat dihitung langsung dengan $n * m$, tidak perlu counter terpisah;
- kedua baris pembatas (garis =) harus memiliki jumlah karakter yang sama seperti pada Contoh Output.

Simpan sebagai latihan_bonus_06.c.

VIII. Pengujian Program

1. Pengujian Program Nested Loop

Program berbasis nested loop harus diuji dari beberapa aspek.

a. Jumlah baris minimum

baris = 1

b. Jumlah kolom minimum

kolom = 1

c. Baris dan kolom sama

baris = 5, kolom = 5

d. Baris lebih banyak daripada kolom

baris = 6, kolom = 2

e. Kolom lebih banyak daripada baris

baris = 2, kolom = 6

f. Nilai batas maksimum

baris = 20, kolom = 20

g. Input tidak valid

baris = 0, kolom = -2

Contoh tabel pengujian pola persegi panjang

No.	Baris	Kolom	Karakter	Expected Result	Status
1	1	1	*	Satu simbol	
2	1	5	#	Satu baris, lima simbol	
3	5	1	X	Lima baris, satu simbol	
4	3	4	+	Tiga baris, empat simbol	
5	0	4	*	Input baris ditolak	
6	4	21	*	Input kolom ditolak	

IX. Tugas Praktikum

1. Tugas Praktikum Modul 6 — Program Rekap Penjualan Beberapa Hari dan Produk

Buat program untuk menerima dan mengolah data penjualan selama beberapa hari. Pada setiap hari terdapat beberapa jenis produk.

Program belum menggunakan array. Data diproses langsung pada saat dimasukkan.

1. Input

Program menerima: jumlah hari pengamatan, jumlah produk, jumlah unit terjual untuk setiap produk pada setiap hari, harga satuan setiap transaksi produk.

Untuk menyederhanakan program, harga satuan dimasukkan pada setiap data penjualan.

2. Batasan Input

- jumlah hari: 1–7;
- jumlah produk: 1–10;
- jumlah unit: 0–1000;
- harga satuan: lebih besar dari 0;
- input yang tidak valid harus diminta kembali.

3. Proses

Untuk setiap produk:

```
subtotal produk = jumlah unit × harga satuan
```

Untuk setiap hari:

```
total harian = jumlah seluruh subtotal produk pada hari tersebut
```

Untuk keseluruhan data:

```
total keseluruhan =  
jumlah seluruh total harian
```

Program juga menghitung:

- jumlah seluruh unit;
- total penjualan per hari;
- hari dengan penjualan tertinggi;
- nilai penjualan tertinggi;
- rata-rata penjualan per hari;
- rata-rata nilai transaksi per data produk.

4. Contoh Interaksi

```
=====
                REKAP PENJUALAN HARIAN DAN PRODUK
=====
Masukkan jumlah hari 1-7      : 2
Masukkan jumlah produk 1-10  : 3

HARI KE-1
-----
Produk ke-1
Jumlah unit : 2
Harga satuan: 10000
Subtotal    : Rp20000.00

Produk ke-2
Jumlah unit : 3
Harga satuan: 5000
Subtotal    : Rp15000.00

Produk ke-3
Jumlah unit : 1
Harga satuan: 20000
Subtotal    : Rp20000.00

Total hari ke-1: Rp55000.00

HARI KE-2
```

```

-----
Produk ke-1
Jumlah unit : 4
Harga satuan: 10000
Subtotal    : Rp40000.00

Produk ke-2
Jumlah unit : 2
Harga satuan: 5000
Subtotal    : Rp10000.00

Produk ke-3
Jumlah unit : 3
Harga satuan: 20000
Subtotal    : Rp60000.00

Total hari ke-2: Rp110000.00

REKAP KESELURUHAN
=====
Jumlah hari           : 2
Jumlah produk per hari : 3
Jumlah seluruh unit   : 15
Total keseluruhan    : Rp165000.00
Rata-rata per hari    : Rp82500.00
Rata-rata per data produk: Rp27500.00
Hari penjualan tertinggi : Hari ke-2
Nilai tertinggi       : Rp110000.00
=====

```

5. Struktur Perulangan yang Wajib Digunakan

Program wajib menggunakan:

- do-while untuk validasi jumlah hari;
- do-while untuk validasi jumlah produk;
- for sebagai perulangan hari;
- for sebagai perulangan produk;
- do-while untuk validasi unit;
- do-while untuk validasi harga;
- accumulator harian;
- accumulator keseluruhan;
- counter jumlah unit;
- seleksi untuk menentukan hari dengan penjualan tertinggi.

6. Tahapan Algoritma

1. meminta jumlah hari;

2. memvalidasi jumlah hari;
3. meminta jumlah produk;
4. memvalidasi jumlah produk;
5. menginisialisasi total keseluruhan;
6. menginisialisasi jumlah seluruh unit;
7. melakukan perulangan untuk setiap hari;
8. menginisialisasi total harian menjadi nol;
9. melakukan perulangan untuk setiap produk;
10. menerima jumlah unit dan harga;
11. memvalidasi data;
12. menghitung subtotal produk;
13. menambahkan subtotal ke total harian;
14. menambahkan unit ke jumlah seluruh unit;
15. setelah seluruh produk selesai, menampilkan total harian;
16. menambahkan total harian ke total keseluruhan;
17. membandingkan total harian dengan nilai tertinggi;
18. setelah seluruh hari selesai, menghitung rata-rata;
19. menampilkan rekap keseluruhan.

7. Kerangka Program

```
#include <stdio.h>

int main(void)
{
    int jumlah_hari;
    int jumlah_produk;
    int hari;
    int produk;
    int jumlah_unit;
    int seluruh_unit = 0;
    int hari_tertinggi = 0;

    double harga_satuan;
    double subtotal;
    double total_harian;
    double total_keseluruhan = 0.0;
    double nilai_tertinggi = 0.0;
    double rata_rata_harian;
    double rata_rata_data_produk;

    do
    {
        printf("Masukkan jumlah hari 1-7: ");
```

```

scanf("%d", &jumlah_hari);

if ((jumlah_hari < 1) || (jumlah_hari > 7))
{
    printf("Jumlah hari tidak valid.\n");
}
}
while ((jumlah_hari < 1) || (jumlah_hari > 7));

do
{
    printf("Masukkan jumlah produk 1-10: ");
    scanf("%d", &jumlah_produk);

    if ((jumlah_produk < 1) ||
        (jumlah_produk > 10))
    {
        printf("Jumlah produk tidak valid.\n");
    }
}
while ((jumlah_produk < 1) ||
        (jumlah_produk > 10));

for (hari = 1; hari <= jumlah_hari; hari++)
{
    total_harian = 0.0;

    printf("\nHARI KE-%d\n", hari);

    for (produk = 1;
        produk <= jumlah_produk;
        produk++)
    {
        /* Validasi jumlah unit. */

        /* Validasi harga satuan. */

        /* Hitung subtotal. */

        /* Tambahkan ke total harian. */

        /* Tambahkan ke seluruh unit. */

        /* Tampilkan subtotal. */
    }

    /* Tambahkan ke total keseluruhan. */
}

```

```

        /* Tentukan hari penjualan tertinggi. */

        /* Tampilkan total harian. */
    }

    /* Hitung rata-rata per hari. */

    /* Hitung rata-rata per data produk. */

    /* Tampilkan rekap. */

    return 0;
}

```

8. Analisis IPO

Mahasiswa wajib melengkapi tabel berikut.

Komponen	Uraian
Input	
Process	
Output	

9. Struktur Perulangan

Mahasiswa wajib menjelaskan fungsi setiap perulangan.

Perulangan	Jenis	Fungsi
Validasi jumlah hari		
Validasi jumlah produk		
Perulangan hari		
Perulangan produk		
Validasi jumlah unit		
Validasi harga satuan		

10. Daftar Accumulator dan Counter

Variabel	Jenis	Diinisialisasi di	Fungsi
total_harian	Accumulator		
total_keseluruhan	Accumulator		
seluruh_unit	Accumulator		
hari	Counter		
produk	Counter		

11. Ketentuan Source Code

Program wajib:

- menggunakan `#include <stdio.h>;`
- menggunakan `int main(void);`
- menggunakan nested `for`;
- menggunakan `do-while` untuk validasi;
- menggunakan accumulator per hari;
- menggunakan accumulator keseluruhan;
- menginisialisasi accumulator pada posisi yang tepat;
- menghitung subtotal produk;
- menghitung total harian;
- menghitung total keseluruhan;
- menentukan hari dengan nilai penjualan tertinggi;
- menghitung rata-rata per hari;
- menghitung rata-rata per data produk;
- memvalidasi seluruh input;
- menggunakan nama variabel deskriptif;
- menggunakan indentasi konsisten;
- dikompilasi dengan warning aktif;
- tidak menghasilkan error atau warning penting;
- diuji menggunakan minimal delapan test case;
- disertai trace table untuk satu data uji;
- dapat dijelaskan oleh mahasiswa.

12. Header Komentar

```
/*
Nama      :
NIM       :
Kelas    :
Modul     : 6
Deskripsi : Program mengolah data penjualan beberapa hari
           dan produk menggunakan perulangan bersarang.
*/
```

13. Berkas yang Dikumpulkan

- source code: `tugas_modul_06_nim.c`
- analisis input–process–output;
- pseudocode atau flowchart;
- penjelasan setiap perulangan;
- daftar counter dan accumulator;
- trace table;
- tabel pengujian minimal delapan kasus;
- tangkapan layar kompilasi;

- tangkapan layar hasil eksekusi;
- catatan debugging;
- refleksi;
- pernyataan penggunaan AI.

2. Contoh Test Case Tugas

No.	Hari	Produk	Data Utama	Expected Result
1	0	–	–	Jumlah hari ditolak
2	8	–	–	Jumlah hari ditolak
3	1	0	–	Jumlah produk ditolak
4	1	11	–	Jumlah produk ditolak
5	1	1	Unit 1, harga 10.000	Total 10.000
6	1	2	2×5.000 dan 3×10.000	Total 40.000
7	2	1	Hari 1: 10.000; Hari 2: 20.000	Tertinggi hari 2
8	2	2	Seluruh unit 0	Total 0
9	2	2	Unit negatif	Input diminta ulang
10	2	2	Harga 0 atau negatif	Input diminta ulang

Untuk data dengan seluruh total harian sama, program dapat mempertahankan hari pertama sebagai hari tertinggi.

3. Diagonal Matriks Perkalian

Buat program C yang membentuk matriks perkalian berukuran $n \times n$ (sel baris ke- i , kolom ke- j berisi $i \times j$) menggunakan *nested loop*, lalu menghitung jumlah nilai pada diagonal utama dan jumlah nilai di luar diagonal. Keluaran program dinilai secara ketat: harus sama persis karakter demi karakter, termasuk spasi.

Format Input

Satu baris bilangan bulat: n (ukuran matriks persegi).

Batasan

- $1 \leq n \leq 15$.

Proses

Untuk setiap sel (i, j) dengan i dan j dari 1 sampai n , nilai sel $= i \times j$. Sel termasuk diagonal utama apabila i sama dengan j .

Contoh 1

Input:

3

Output:

```
=====
DIAGONAL MATRIKS PERKALIAN
=====
```

```

Ukuran matriks      : 3
-----
Total diagonal       : 14
Total non-diagonal   : 22
Total keseluruhan   : 36
=====

```

Contoh 2

Input:

4

Output:

```

=====
          DIAGONAL MATRIKS PERKALIAN
=====
Ukuran matriks      : 4
-----
Total diagonal       : 30
Total non-diagonal   : 70
Total keseluruhan   : 100
=====

```

Contoh 3

Input:

1

Output:

```

=====
          DIAGONAL MATRIKS PERKALIAN
=====
Ukuran matriks      : 1
-----
Total diagonal       : 1
Total non-diagonal   : 0
Total keseluruhan   : 1
=====

```

Ketentuan

- gunakan nested for dengan counter i (baris) dan j (kolom), keduanya dari 1 sampai n;
- gunakan kondisi if (i == j) di dalam nested loop untuk memisahkan akumulasi diagonal dan non-diagonal;
- Contoh 3 menunjukkan kasus n = 1 — satu-satunya sel adalah diagonal, sehingga total non-diagonal harus 0;
- total keseluruhan harus sama dengan total diagonal ditambah total non-diagonal;
- kedua baris pembatas (garis =) harus memiliki jumlah karakter yang sama seperti pada Contoh Output;
- keluaran diperiksa apa adanya (karakter demi karakter), sehingga kesalahan satu spasi tetap dianggap tidak sesuai.

Simpan sebagai tugas_modul_06_soal2_nim.c dan sertakan sebagai lampiran tambahan bersama tugas utama.

X. Kesalahan Umum dan Standar Penulisan

1. Kesalahan yang Sering Terjadi

a. *Perulangan dalam tidak diinisialisasi ulang*

Salah:

```
int kolom = 1;

for (baris = 1; baris <= 3; baris++)
{
    while (kolom <= 4)
    {
        printf("*");
        kolom++;
    }
}
```

Setelah baris pertama, kolom sudah bernilai 5. Baris berikutnya tidak menjalankan perulangan dalam.

Perbaiki:

```
for (baris = 1; baris <= 3; baris++)
{
    kolom = 1;

    while (kolom <= 4)
    {
        printf("*");
        kolom++;
    }
}
```

b. *Accumulator per kelompok tidak diatur ulang*

Salah:

```
double total_harian = 0.0;

for (hari = 1; hari <= jumlah_hari; hari++)
{
    for (produk = 1; produk <= jumlah_produk; produk++)
    {
        total_harian += subtotal;
    }
}
```

Perbaiki:

```
for (hari = 1; hari <= jumlah_hari; hari++)
{
    total_harian = 0.0;

    for (produk = 1; produk <= jumlah_produk; produk++)
```

```

    {
        total_harian += subtotal;
    }
}

```

c. Total keseluruhan diatur ulang di dalam loop

Salah:

```

for (hari = 1; hari <= jumlah_hari; hari++)
{
    total_keseluruhan = 0.0;
    total_keseluruhan += total_harian;
}

```

Nilai dari hari sebelumnya hilang.

d. `printf("\n")` ditempatkan di lokasi yang salah

Apabila berada di dalam perulangan dalam:

```

for (kolom = 1; kolom <= 4; kolom++)
{
    printf("*");
    printf("\n");
}

```

setiap simbol akan berada pada baris terpisah.

e. Menggunakan counter yang salah

Salah:

```

for (kolom = 1; kolom <= baris; kolom++)
{
    printf("%d ", baris);
}

```

Ini benar apabila ingin mengulang nomor baris. Namun, salah apabila targetnya:

```
1 2 3 4
```

Untuk tujuan tersebut, tampilkan:

```
printf("%d ", kolom);
```

f. Batas nested loop salah

```
for (baris = 1; baris < 5; baris++)
```

hanya menghasilkan empat baris.

g. Update bergerak ke arah yang salah

```
for (kolom = 1; kolom <= 5; kolom--)
```

berpotensi menyebabkan infinite loop.

h. `break` dianggap menghentikan seluruh loop

`break` hanya menghentikan loop terdekat.

i. Tidak memvalidasi pembagi

```
rata_rata =
    total_keseluruhan /
    (jumlah_hari * jumlah_produk);
```

Pastikan jumlah hari dan jumlah produk lebih besar dari nol.

j. Menimpa total

Salah:

```
total_harian = subtotal;
```

Benar:

```
total_harian += subtotal;
```

2. Standar Penulisan Nested Loop***Gunakan nama counter sesuai konteks***

Disarankan:

```
int baris;
int kolom;
```

atau:

```
int hari;
int produk;
```

Daripada:

```
int i;
int j;
```

Untuk contoh matematis sederhana, *i* dan *j* masih dapat digunakan. Untuk kasus nyata, nama kontekstual lebih mudah dipahami.

Gunakan indentasi bertingkat

```
for (baris = 1; baris <= tinggi; baris++)
{
    for (kolom = 1; kolom <= baris; kolom++)
    {
        printf("* ");
    }

    printf("\n");
}
```

Indentasi menunjukkan hubungan antara perulangan luar dan dalam.

Hindari nested loop terlalu dalam

Tiga atau lebih tingkat perulangan dapat sulit dipahami oleh mahasiswa semester 1.

```

for (...)
{
    for (...)
    {
        for (...)
        {
            for (...)
            {
                /* ... */
            }
        }
    }
}

```

Apabila masalah memerlukan struktur sangat dalam, pertimbangkan dekomposisi menjadi fungsi setelah Modul 7.

Pisahkan accumulator

```

double total_harian;
double total_keseluruhan = 0.0;

```

Nama harus memperlihatkan cakupan hasil.

Gunakan komentar pada tingkat proses

```

// Mengolah seluruh produk pada satu hari.
for (produk = 1; produk <= jumlah_produk; produk++)
{
    /* ... */
}

```

Komentar tidak perlu menjelaskan sintaks secara harfiah.

XI. Pernyataan Penggunaan AI

Tidak Menggunakan AI:

Saya menyatakan bahwa tugas ini dikerjakan tanpa menggunakan bantuan AI generatif.

Menggunakan AI:

Saya menggunakan [nama perangkat AI] untuk membantu menjelaskan [nested loop, error, warning, test case, atau bagian lain]. Hasil tersebut saya periksa melalui tracing, kompilasi, debugging, pengujian batas, dan perbandingan dengan algoritma yang disusun.

Daftar Pustaka

Deitel, P., & Deitel, H. (2016). C: How to program (8th ed.). Pearson Education.

Free Software Foundation. (n.d.). GCC, the GNU Compiler Collection: Online documentation. GNU Project. Diakses dari <https://gcc.gnu.org/onlinedocs/>

- Free Software Foundation. (n.d.). GDB: The GNU Project debugger. GNU Project. Diakses dari <https://www.gnu.org/software/gdb/>
- International Organization for Standardization. (2018). ISO/IEC 9899:2018 — Information technology — Programming languages — C. ISO.
- Kernighan, B. W., & Ritchie, D. M. (1988). The C programming language (2nd ed.). Prentice Hall.
- McConnell, S. (2004). Code complete: A practical handbook of software construction (2nd ed.). Microsoft Press.
- Microsoft. (n.d.). C/C++ for Visual Studio Code. Visual Studio Code Documentation. Diakses dari <https://code.visualstudio.com/docs/languages/cpp>